PRACTICAL NO. 9

AIM: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2,3,4, L,2,5, !,2,3,4, 5 (No of Frames=3)

CODE:

MINGW64:/d/OS_Lab

GNU nano 8.7 optimal.c

```c
#include <stdio.h>

int main() {
    int frames = 3;
    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    int n = 12;

    int frame[3];
    int i, j, k, pos, faults = 0, hits = 0;

    // Initialize frames with -1
    for(i = 0; i < frames; i++)
        frame[i] = -1;

    printf("\nPage Replacement Process:\n");

    for(i = 0; i < n; i++) {
        int found = 0;

        // Check for hit
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                hits++;
                break;
            }
        }

        if(found == 0) {
            faults++;

            // Find empty frame
            int empty = -1;
            for(j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    empty = j;
                    break;
                }
            }
```

MINGW64:/d/OS_Lab

GNU nano 8.7 optimal.c

```
                frame[empty] = pages[i];
            } else {
                // Optimal Replacement Logic
                int farthest = -1, index = -1;

                for(j = 0; j < frames; j++) {
                    int found_future = 0;
                    for(k = i + 1; k < n; k++) {
                        if(frame[j] == pages[k]) {
                            if(k > farthest) {
                                farthest = k;
                                index = j;
                            }
                            found_future = 1;
                            break;
                        }
                    }

                    if(found_future == 0) {
                        index = j;
                        break;
                    }
                }

                frame[index] = pages[i];
            }
        }

        // Print frame status
        printf("\nPage %d -> ", pages[i]);
        for(j = 0; j < frames; j++) {
            if(frame[j] != -1)
                printf("%d ", frame[j]);
            else
                printf("- ");
        }

        if(found)
            printf(" (Hit)");
        else
            printf(" (Fault)");
    }

    printf("\n\nTotal Page Hits = %d", hits);
    printf("\nTotal Page Faults = %d", faults);

    float hit_ratio = (float)hits / n * 100;
    float fault_ratio = (float)faults / n * 100;

    printf("\nHit Percentage = %.2f%%", hit_ratio);
    printf("\nFault Percentage = %.2f%%\n", fault_ratio);

    return 0;
}
```

OUTPUT:

```
MINGW64:/d/OS_Lab
Haris@Areeba-PC MINGW64 /d/OS_Lab (main)
$ nano optimal.c

Haris@Areeba-PC MINGW64 /d/OS_Lab (main)
$ gcc optimal.c -o optimal

Haris@Areeba-PC MINGW64 /d/OS_Lab (main)
$ ./optimal

Page Replacement Process:

Page 1 -> 1 - -  (Fault)
Page 2 -> 1 2 -  (Fault)
Page 3 -> 1 2 3  (Fault)
Page 4 -> 1 2 4  (Fault)
Page 1 -> 1 2 4  (Hit)
Page 2 -> 1 2 4  (Hit)
Page 5 -> 1 2 5  (Fault)
Page 1 -> 1 2 5  (Hit)
Page 2 -> 1 2 5  (Hit)
Page 3 -> 3 2 5  (Fault)
Page 4 -> 4 2 5  (Fault)
Page 5 -> 4 2 5  (Hit)

Total Page Hits = 5
Total Page Faults = 7
Hit Percentage = 41.67%
Fault Percentage = 58.33%

Haris@Areeba-PC MINGW64 /d/OS_Lab (main)
```